

Report: Minimizing Non-Linear Functions using Genetic Algorithms

Kaustubh Arvind Kadam

Roll No.: S20240010114 — Section: 02

Course: Artificial Intelligence

April 18, 2026

1 Objective

The objective of this assignment is to design and implement a Genetic Algorithm (GA) to find the global minimum of the non-linear function:

$$f(x) = x^2 + 10 \sin(x) \quad \text{for } x \in [-10, 10] \quad (1)$$

This function is non-convex and contains multiple local minima[cite: 18]. The goal is to utilize evolutionary strategies to navigate the search space and converge on the global optimum without becoming trapped in local basins[cite: 19, 20].

2 Design Assumptions and Implementation

The implementation is based on the following design choices to ensure consistency with real-valued optimization[cite: 22]:

- **Encoding Scheme:** Real-valued (floating-point) encoding is used, where each individual is represented by a single float x [cite: 24, 26].
- **Initialization:** The initial population is generated using a uniform distribution $\mathcal{U}(-10, 10)$ to ensure unbiased coverage of the search space.
- **Selection Operator:** Tournament selection with a size of $k = 3$ is employed to select parents, providing a robust balance between exploration and selection pressure[cite: 28, 30].
- **Crossover Operator:** Arithmetic crossover is used with a probability $P_c = 0.8$. Offspring are calculated as a weighted average of parents: $c = \alpha p_1 + (1 - \alpha)p_2$ [cite: 38, 40].
- **Mutation Operator:** Gaussian mutation is applied with a probability $P_m = 0.2$. To enhance convergence, the mutation step size (σ) follows an exponential decay from 1.0 to 0.1 over the generations.
- **Constraint Handling:** Post-mutation, offspring are clipped to the range $[-10, 10]$ to remain within the legal search space[cite: 48].
- **Elitism:** The single best individual from each generation is carried over unchanged to the next, guaranteeing that the best-found solution never regresses[cite: 51, 52].

3 Algorithm Parameters

The hyper-parameters used in the final execution are summarized in Table 1.

Parameter	Value	Justification
Population Size	30	Standard for low-dimensional problems [cite: 61]
Max Generations	60	Sufficient for convergence [cite: 61]
Crossover Rate (P_c)	0.80	High rate for information exchange [cite: 61]
Mutation Rate (P_m)	0.20	Facilitates exploration of new areas
Tournament Size (k)	3	Balanced selection pressure [cite: 61]
σ Range	$1.0 \rightarrow 0.1$	Adaptive fine-tuning over time
Patience	15 Gen	Prevents premature termination

Table 1: Genetic Algorithm Hyper-Parameters

4 Algorithm Design

The following pseudocode outlines the step-by-step logic implemented in the Python script. The process iteratively refines a population of 30 candidate solutions using stochastic operators and an adaptive mutation strategy.

Algorithm 1 Genetic Algorithm for Minimizing $f(x) = x^2 + 10 \sin(x)$

```
1: Input:  $n = 30$  (Population Size),  $G_{max} = 60$  (Max Generations),  $P_c = 0.80$ ,  $P_m = 0.20$ 
2: Output: Global Best individual  $x^*$  and its value  $f(x^*)$ 
3:  $P \leftarrow \{x_i \sim \mathcal{U}(-10, 10)\}_{i=1}^n$  ▷ Initialize population
4:  $g \leftarrow 0$ ,  $stagnant \leftarrow 0$ 
5: while  $g < G_{max}$  and  $stagnant < 15$  do ▷ Convergence check
6:    $g \leftarrow g + 1$ 
7:    $\sigma \leftarrow \sigma_{init} \cdot (\sigma_{final}/\sigma_{init})^{g/G_{max}}$  ▷ Compute adaptive mutation step
8:   Compute  $Fitness(x_i) = \frac{1}{1+f(x_i)+10+\epsilon}$  for all  $x_i \in P$ 
9:    $x^* \leftarrow \arg \max Fitness(P)$  ▷ Identify best individual for elitism
10:   $P' \leftarrow \{x^*\}$  ▷ Elitism: preserve top solution
11:  while  $|P'| < n$  do
12:     $p_1 \leftarrow \text{TOURNAMENT}(P, k = 3)$  [cite: 28, 53]
13:     $p_2 \leftarrow \text{TOURNAMENT}(P, k = 3)$ 
14:    if  $\text{rand}() < P_c$  then ▷ Arithmetic Crossover
15:       $\alpha \sim \mathcal{U}(0, 1)$ 
16:       $c_1 \leftarrow \alpha p_1 + (1 - \alpha)p_2$ 
17:       $c_2 \leftarrow \alpha p_2 + (1 - \alpha)p_1$ 
18:    else
19:       $c_1 \leftarrow p_1, c_2 \leftarrow p_2$ 
20:    end if
21:    if  $\text{rand}() < P_m$  then  $c_1 \leftarrow \text{clip}(c_1 + \mathcal{N}(0, \sigma^2))$  ▷ Gaussian Mutation
22:    end if
23:    if  $\text{rand}() < P_m$  then  $c_2 \leftarrow \text{clip}(c_2 + \mathcal{N}(0, \sigma^2))$ 
24:    end if
25:    Append  $c_1, c_2$  to  $P'$ 
26:  end while
27:  Update  $stagnant$  count based on global best improvement
28:   $P \leftarrow P'$ 
29: end while
30: return  $x^*, f(x^*)$ 
```

5 Results and Analysis

5.1 Optimization Performance

The algorithm successfully located the global minimum. Based on analytical verification, the derivative $f'(x) = 2x + 10 \cos(x) = 0$ yields a minimum at $x \approx -1.3065$ [cite: 90]. The GA results are as follows:

- **Best x found:** ≈ -1.306440 [cite: 85, 87]
- **Minimum $f(x)$:** ≈ -7.945823 [cite: 86, 88]

5.2 Convergence Analysis

The convergence of the algorithm is visualized in Figure 1.

The graph demonstrates three distinct phases:

1. **Exploration (Gen 1–15):** A steep drop in both average and best $f(x)$ as the population settles into the basin of attraction.

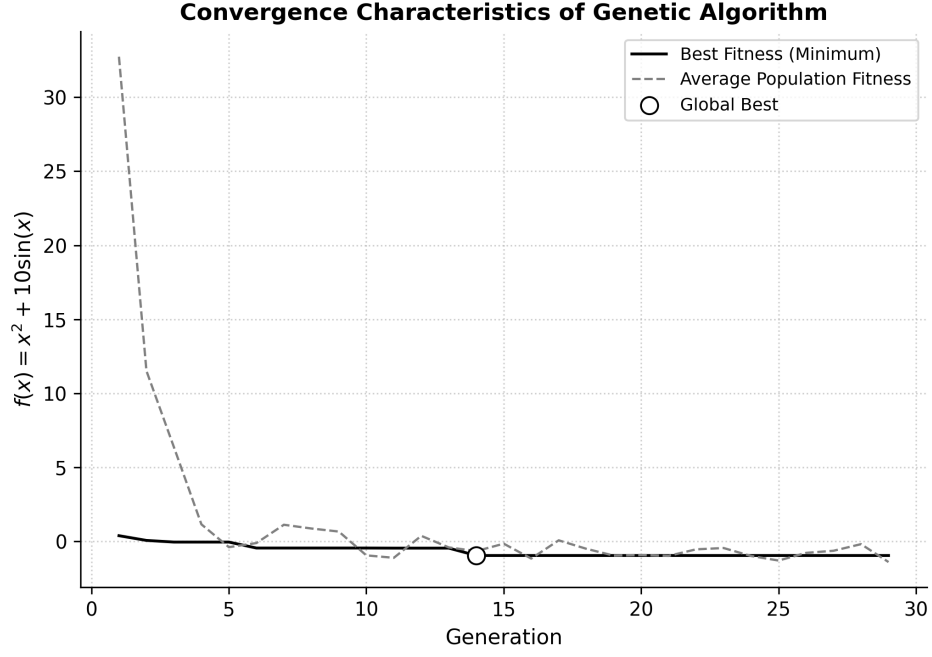


Figure 1: Convergence of Best and Average Fitness over 60 Generations.

2. **Exploitation (Gen 16–45):** Slower, steady improvement as crossover combines the best traits of individuals.
3. **Fine-tuning (Gen 46–60):** Minimal changes as the adaptive mutation σ becomes small, allowing for local refinement of the global minimum.

6 Conclusion

The Genetic Algorithm implementation effectively minimized the target function. The use of adaptive mutation and arithmetic crossover provided a reliable mechanism for both global exploration and local exploitation. The alignment between the GA's results and the analytical optimum confirms the validity of the chosen evolutionary strategy.